

Dual-Teacher and Dual-Prompt Pool for Few-Shot Dialog State Tracking

Di Wu[✉], Yuheng Li[✉], and Zhizhi Yu[✉]

Abstract—Dialog state tracking (DST) is an essential component of task-oriented dialog (ToD) systems. Existing few-shot DST models face challenges in effectively leveraging semantically related information and exhibit limited adaptability to dialog scenarios. To address these challenges, the dual-teacher and dual-prompt pool model for few-shot DST (DDP-DST) is proposed. Specifically, by enhancing key semantic information and syntactic structure, dual teacher models are constructed to generate pseudolabels from complementary perspectives. Self-training is employed to further improve state value generation. Besides, considering the intrinsic symmetry between slot and state value generation tasks, a dual-prompt fine-tuning strategy is designed. A dynamic prompt pool is constructed to adaptively generate prompts. Reconstruction errors (REs) are fed back into the DDP-DST model, leading to improved accuracy in DST. Experimental results on the MultiWOZ 2.1 dataset demonstrate that DDP-DST outperforms baseline models such as SM2-3b, DS2, and SVAG with average improvements of 4.3%, 2.4%, and 2.0% in the metric of joint goal accuracy (JGA). Notably, with fewer than 1 billion parameters, DDP-DST achieves competitive performance in few-shot settings, even outperforming models with up to 10 billion parameters.

Index Terms—Dialog state tracking (DST), dual-prompt pool, dual-teacher self-training (DS), few-shot learning, task-oriented dialog (ToD).

I. INTRODUCTION

THE emergence of generative artificial intelligence has injected new vitality into human-computer interaction. Task-oriented dialog (ToD) as a crucial branch of human-computer interaction, enables users to access specialized services across various domains [1]. Dialog state tracking (DST) aims to identify the slots and state values in each round of dialog, forming the cornerstone of understanding user intentions in ToD, enabling systems to provide accurate and contextually appropriate responses.

Training a DST model requires integrating dialog history, context, and previous states to infer and update the current state. However, obtaining round-level labeled dialog states is resource-intensive, limiting the development of robust models. Researchers have explored several few-shot approaches to

address this challenge. Some methods [2], [3] design specific mechanisms to reduce reliance on manual labeling. However, these mechanisms often struggle to capture deeper semantic information. Other methods [4], [5] fine-tune pretrained language models (PLMs) to adapt to few-shot tasks and reduce model complexity. Yet, these approaches require retraining when encountering new slots or domains, reducing their practical applicability in dynamic dialog environments.

To address the ineffective utilization of semantically related information and the limited adaptability of dialog scenarios in existing models, we propose a dual-teacher and dual-prompt pool model for few-shot DST (DDP-DST). By enhancing key dialog information and extending contextual syntactic diversity, dialog state values are iteratively generated through self-training, improving the semantic comprehension ability. Additionally, a dynamic prompt pool is constructed to generate specific prompts based on the context. By minimizing reconstruction error (RE), the accuracy of few-shot DST is ultimately enhanced. Experiments conducted on the MultiWOZ 2.1 [6] dataset verify the effectiveness of the proposed method.

The main contributions of this article are as follows.

- 1) This article presents the DDP-DST model, pioneering the use of a dual-teacher self-training (DS) method for the DST task.
- 2) A dynamic dual-prompt (DD) construction method is proposed to adapt to the few-shot scenario. In addition, a novel RE feedback mechanism is designed for the dual task framework.
- 3) Experimental results demonstrate that the proposed DDP-DST model outperforms the compared methods in assessments of the few-shot DST task. The validity of the components in the model is verified through ablation experiments and a series of analyses.

II. RELATED WORK

A. Semi-Supervised Self-Training Method

Self-training is a commonly used semi-supervised learning strategy that utilizes pseudolabels of unlabeled samples [7]. It was first introduced to the DST task by Zhang et al. [8]. Self-supervised and self-training methods are integrated into tracking states. The LAVE self-training framework was proposed by Lee et al. [9]. It combines pseudolabeling and validation models without relying on fixed thresholds. The field is further advanced by Gu et al. [10]. A state value

Received 19 February 2025; revised 1 August 2025 and 4 November 2025; accepted 26 January 2026. Date of publication 18 February 2026; date of current version 2 September 2026. This work was supported in part by the Research Projects of the Natural Science Foundation of Hebei Province under Grant F2020402003. (Corresponding author: Di Wu.)

The authors are with the School of Information and Electrical Engineering, Hebei University of Engineering, Handan 056000, China (e-mail: wudiwudi@hebeu.edu.cn; lyh_32_1@163.com; yuzhizhi2000@gmail.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNNLS.2026.3659341>, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2026.3659341

2162-237X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09, 2026 at 12:47:05 UTC from IEEE Xplore. Restrictions apply.

evaluator is designed using a negative sampling method. The accuracy of state value generation is improved during the self-training process. The above method enhances the utilization of unlabeled samples. However, the feature knowledge is primarily learned from a single perspective, underutilizing the semantic information inherent in unlabeled samples.

DS methods can learn data features from different perspectives. The dual-teacher framework for cross-domain ship detection was proposed by Zheng et al. [11]. Cross-domain data are used to train the two teacher models by a cotraining strategy. The DTBS framework for night-time semantic segmentation was proposed by Huang et al. [12]. The teacher models are divided into illumination and style categories, which allows them to learn the specific features of the target domain. The quality of pseudolabel generation is enhanced by the above studies using dual-teacher architecture, which is also highly compatible with the requirements of the few-shot DST task. In this article, we apply the DS method for state value generation. Pseudolabels are generated from two data enhancement perspectives to capture both the syntactic structure and key information features of unlabeled samples.

B. Dynamic Prompt Generation

With the widespread adoption of PLMs, prompt learning has emerged as a dominant paradigm in natural language processing (NLP) due to its exceptional capabilities in information understanding and task adaptation [13]. Prompt learning methods can be broadly categorized into static prompting and dynamic prompting. Static prompting typically involves manually designing fixed prompt templates. It may suffer from repetitive or incomplete errors. In contrast, dynamic prompting adjusts the content of the prompts based on the context of the current dialog or task, guiding the model to generate outputs that better meet the desired expectations [14].

To adapt to different task scenarios, the first automated prompt generation method was proposed by Shin et al. [15]. Original task input and trigger words learned by a gradient search strategy are combined to construct dynamic prompts. An automatically generated prompts framework was proposed by Zhao et al. [16]. It automatically constructs relation-specific prompts by combining relation descriptions and contextual clues, enabling the model to dynamically adapt its input representation for different relation types. In the field of ToD, a closed-box prompt learning approach was proposed by Hu et al. [17]. It creates dynamic prompt optimizers to generate responses that are more aligned with user preferences. Dynamic prompts are generated using tabularized domain descriptions. A keyword prompt generation method is designed by Qixiang et al. [18]. It extracts verbs and nouns related to domain slots, randomly sampled to integrate into dynamic prompts. A retrieval mechanism was designed by Chen et al. [19]. It filters context examples from the candidate pool to generate prompts. A prompt pool-based class-incremental continual learning method was proposed by Liu et al. [20]. It uses a pool of prompt-key pairs to select relevant prompts based on input context, facilitating task identification and state prediction. Given the advancements in these methods, which utilize dynamic prompt learning to incorporate

contextual information and enhance task performance, we explore the application of the prompt pool method to the few-shot DST task, enabling the model to dynamically adapt to context with limited data.

C. Few-Shot DST

Few-shot DST requires the ability to generalize state features from limited labeled data, making the efficient mining of semantic information a key challenge. Early research studies primarily relied on rule-based and statistics-based approaches. However, manually summarized rules can hardly cover different dialog domains, mathematical statistical models are susceptible to noise and struggle to extract potential features [21]. Currently, for the capability of capturing intricate semantic structures and contextual dependencies, PLMs are emerging as a dominant paradigm in few-shot DST [22].

Many researchers have incorporated fine-tuning into few-shot DST, adjusting pretrained model parameters to address downstream task requirements better. The FaS-DST framework was proposed by Feng et al. [23]. It decomposes the DST task into three subtasks. Dialog summaries are predicted to assist in generating dialog states. A dual-prompt learning framework was constructed by Yang et al. [5]. It is based on the dual structure of the DST task. A prompt integration template is designed to improve accuracy. DST was decomposed into state value generation and domain slot generation subtasks by Gu et al. [10]. Prompt learning and Self-training techniques are combined for stage-wise DST. Few-shot domain transfer is achieved through lightweight parameter tuning using the above methods. However, they rely on static rules or hand-designed mechanisms, incompatible with complex dialog domains. In this article, a dual-prompt learning fine-tuning strategy is designed. Adaptation to the dialog domain is achieved through constructing a dynamic prompt pool and minimizing RE.

III. DDP-DST MODEL

To address the issues of ineffective semantic information utilization and limited adaptability of few-shot DST models, the DDP-DST is proposed. First, pseudolabels are alternately generated from two complementary perspectives: key semantic information enhancement and syntactic structure enhancement. After filtering high-confidence pseudolabels, the student model learns fused knowledge from both teachers, and its parameters are smoothly updated using EMA. Then, a closed loop of slot generation and state value generation dual tasks is constructed. A dynamic prompt pool is created to filter and select state-associated key prompts. The text-to-text transfer transformer (T5) model [24] is dynamically guided to generate both slots and state values. By minimizing the RE between generated and ground-truth slots, the model establishes feedback signals that constrain and refine the T5 model's outputs. The framework of the DDP-DST model is illustrated in Fig. 1.

A. DS State Value Generation

In few-shot scenarios, traditional self-training methods learn dialog features from a single perspective, leading to insufficient utilization of the implicit semantic information in

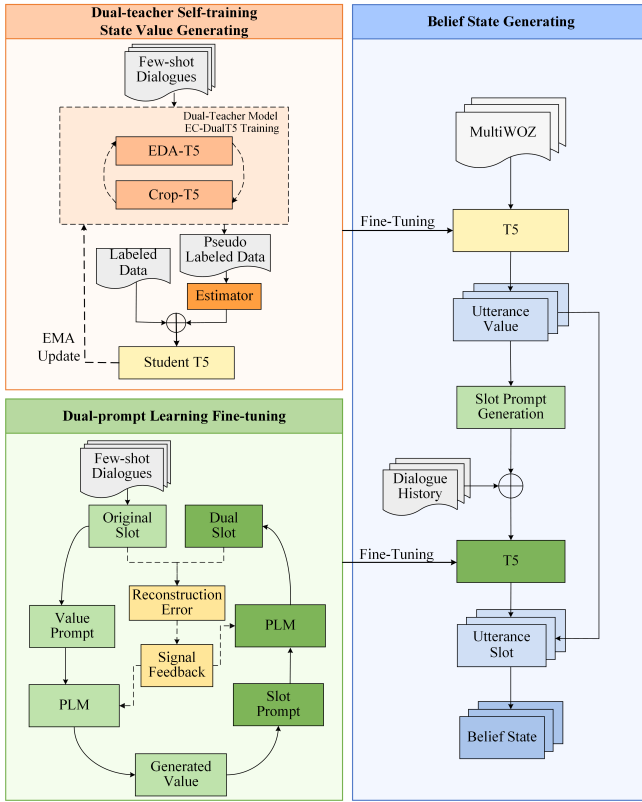


Fig. 1. DDP-DST model framework.

unlabeled samples. To address this challenge, a DS state value generation method is proposed. By employing key information enhancement and syntactic structure enhancement strategies, two teacher models, EDA-T5 and Crop-T5, are trained separately to form the dual-teacher model dual teacher T5 model with EDA and crop augmentation (EC-DualT5). The StudentT5 learns from the fusion of dual-perspective pseudolabel knowledge. The implicit semantics and distribution characteristics of state values are captured in different contexts. Parameters are smoothly updated with feedback from the EC-DualT5 model.

The EDA method [25] enriches syntactic diversity through rule-based paraphrasing. It enhances expression styles while maintaining semantic integrity. The Crop method [26] reduces redundant components in sentences, emphasizing the core semantic information relevant to the task. Given the complementary nature of the EDA and the Crop method in terms of syntax and semantics, the two methods are selected as the enhancement strategies for the EC-DualT5 model. Abnormal dialog data is cleaned by regular matching and replacement. Incomplete dialogs and incorrectly labeled data are removed to reduce the impact of noise. Synonym replacement (SR), random insertion (RI), random deletion (RD), and random swap (RS) are randomly selected in each sample to rewrite user utterances.

To keep our method simple and language-agnostic, we adopt a rule-based design for the interest dependency list used in our crop-based pruning. This list includes core universal dependency relations such as nominal subject (nsubj), indirect object

(iobj), object (obj), and oblique nominal (obl), as proposed by Şahin and Steedman [26]. These labels guide the identification of meaningful syntactic chunks to retain during cropping. In addition, we follow their approach to handle multiword predicates using the relations fixed, flat, cop, and compound. To avoid a combinatorial explosion, reordering is only applied at the first level of the dependency tree, and modifiers like advmod are omitted for simplicity. For example, consider the sentence, “Can you also find me an entertainment attraction near Bloomsbury.” The syntactic structure and dependency relationships are analyzed, as shown in Fig. 2.

In Fig. 2, the direction of the arrows indicates the dependency relationships from the dependent words to the headwords. The labels marked on the arrows, such as aux (auxiliary verb) and iobj, represent specific syntactic dependency relationship types. For example, “entertainment” points to “attraction,” indicating a compound relationship where “attraction” depends on “entertainment.” By removing dependencies unrelated to the core information, a concise sentence focusing on key information is obtained: “You find an entertainment attraction near Bloomsbury.” The Crop method rewrites user utterances in each round, retaining the most semantically complete version.

To learn state value features from different perspectives, augmentation strategies are employed to construct datasets for the dual-teacher model EC-DualT5. During training, the input sequence X_i for the i th turn is shown below

$$X_i = \text{Encoder}(\text{PREFIX} \oplus D_i) \quad (1)$$

where PREFIX denotes the predefined prefix prompt “Get the requests that the user confirmed or mentioned in this turn.” Encoder($\cdot$) denotes the encoding process of the Transformer. $\oplus$ is the concatenation operation of sequences. $D_i = \{u_i, r_i, D_{i-1}\}$ is the dialog history. u_i represents the user utterance in the i th turn, r_i denotes the system response in the i th turn. D_{i-1} denotes the dialog history up to the $(i-1)$ th turn.

By decoding X_i , a state value sequence $\hat{V}_i = \{\hat{v}_1, \hat{v}_2, \dots, \hat{v}_n\}$ composed of n state values is obtained. The equation of $\hat{V}_i$ is shown below

$$\mathcal{L}_{\text{self}} = - \sum \log P(V_i | D_i; \text{PREFIX}) \quad (2)$$

where $P(V_i | D_i; \text{PREFIX})$ denotes the probability distribution of generating state values V_i given the dialog history D_i and PREFIX. V_i denotes the true state value labels.

After the EC-DualT5 model is trained, pseudolabels are generated for the remaining unlabeled samples. Since the RoBERTa model [27] generates dynamic masks and achieves exceptional performance in inference tasks, a RoBERTa-based evaluation mechanism is constructed. To ensure domain consistency between the evaluator and the downstream DST task, the RoBERTa-based classifier is fine-tuned using the same few-shot dataset as the DDP-DST model. This allows the classifier to more effectively assess pseudolabel quality under the same distribution. The evaluation of pseudolabels is treated as a three-class classification task, predicting the probabilities of correct, incomplete, and incorrect generation.

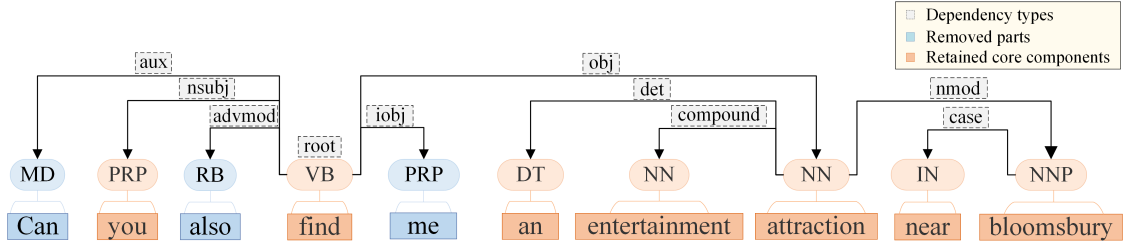


Fig. 2. Example of the crop method.

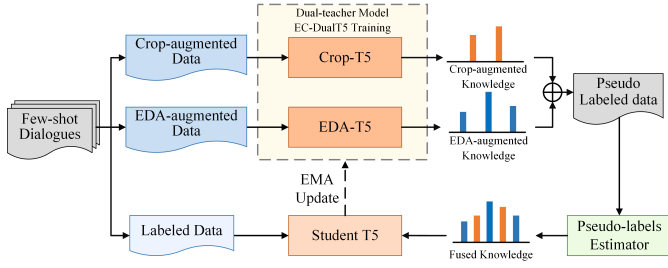


Fig. 3. Training framework of the EC-DualT5 model.

To construct a negative sample dataset, state values are randomly removed and readdd to the dialog data. The equation of the evaluation loss $\mathcal{L}_{\text{eval}}$ is shown below

$$\mathcal{L}_{\text{eval}} = - \sum_{i=1}^C y_i \log(\hat{p}_i) \quad (3)$$

where C is the number of classification categories. y_i is the true label for the i th category. $\hat{p}_i$ is the predicted probability of the sample belonging to the i th category.

The probability that the pseudolabel belongs to the correct generation category is used as the confidence measure of pseudolabel quality. We retain the pseudolabel with the highest confidence score among those surpassing the threshold ($P_r = 0.98$) to construct the pseudolabel dataset. Then, we use it to retrain the StudentT5. The training framework of the EC-DualT5 model is shown in Fig. 3.

In Fig. 3, the StudentT5 integrates the combined feature knowledge from the EC-DualT5 and feeds the learned parameter weights back. To stabilize the training process, the EMA mechanism is introduced to smooth the weight updates and avoid large fluctuations. Taking the EDA-T5 model as an example, the equation of the parameter $\theta_{\text{teacher}}^n$ for the n th EMA update is shown below

$$\theta_{\text{teacher}}^n = \alpha \cdot \theta_{\text{teacher}}^{n-1} + (1 - \alpha) \cdot \theta_{\text{student}}^{n-1} \quad (4)$$

where $\theta_{\text{teacher}}^{n-1}$ is the parameters of the EDA-T5 model at the $(n-1)$ th epoch. $\theta_{\text{student}}^{n-1}$ is the parameters of the StudentT5 at the $(n-1)$ th epoch. $\alpha \in (0, 1)$ denotes the decay coefficient, which controls the influence weight of the model parameters.

The StudentT5 gradually adjusts its parameters during each training epoch to align with the EC-DualT5 model. It maintains the original multiperspective feature knowledge from EC-DualT5 while incorporating newly learned integrated knowledge from its training.

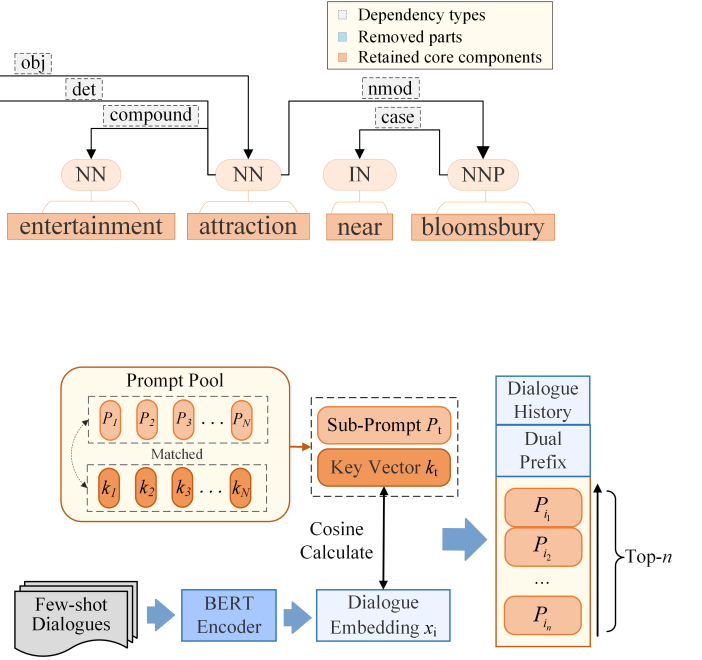


Fig. 4. DD construction.

Through the EDA method, the semantic meaning of the context is preserved while enriching sentence structure, enabling the model to learn diverse ways of expressing sentences. The Crop method trims irrelevant sentence components. Slot-related information is retained to focus on learning key state value features. Model parameters are smoothly updated to provide feedback to the EC-DualT5 model, enhancing the quality of pseudolabel generation.

B. Dual-Prompt Learning Fine-Tuning

Potential state value features in few-shot scenarios are captured after DS state value generation. However, complex dependencies between state values and slot types make it difficult to generate slots accurately. Therefore, the dual-prompt learning fine-tuning strategy is designed. By utilizing DDs, the semantic mapping between state values and slots is captured to enhance the adaptability of the dialog scenarios. RE signals guide the DDP-DST model in understanding the semantic relationship between slots and state values.

1) *DD Constructing*: Different dialog scenarios across domains exhibit variations in semantic expressions, posing challenges for traditional dual-learning frameworks. Therefore, the DD construction method is proposed. It generates prompts for dual tasks to guide the DDP-DST model through a closed-loop feedback process, enabling adaptive adjustments to dialog scenarios and dual tasks. The construction of DDs is illustrated in Fig. 4.

In Fig. 4, a dynamic prompt pool $P = \{(k_i, P_i)\}_{i=1}^N$ is constructed, where P_i represents the i th subprompt. $k_i \in \mathbb{R}^{L_K}$ denotes the learnable key vector assigned to each subprompt vector with a length of L_K . K represents the set of all key vectors. N is the number of subprompts contained in the prompt pool. The arrows point from the prompt pool to the key

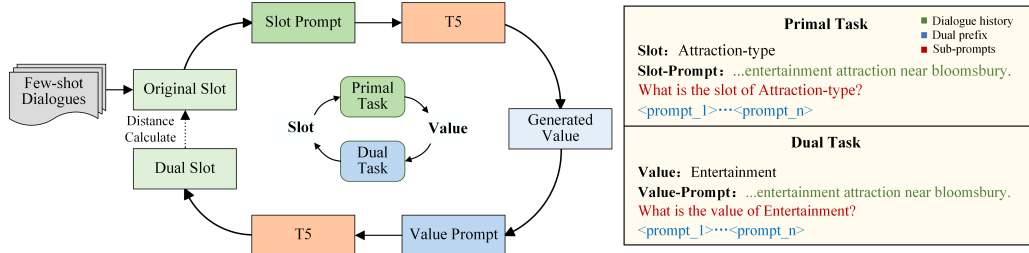


Fig. 5. Dual-generation task loop.

vector k_t and subprompt P_t indicates the specific content of the candidate prompts. Top- n denotes the selected n subprompts closest to the t th turn dialog embedding x_t based on cosine similarity.

The dialog context $C_i = \{u_i, r_i\}$ for the turn i th are extracted from the few-shot dialog dataset. u_i is the user utterance and r_i denotes the system response. bidirectional encoder representations from transformers (BERTs) [28] is used for encoding to transform dialog context C_i into high-dimensional vector representations $x_i \in \mathbb{R}^{L_K}$.

The cosine similarity between the vector x_i and k_t is calculated to form a prompt sequence M_i in the i th turn of the dialog. The equation of M_i is shown below

$$M_i = D_i \oplus \text{PREFIX}_{\text{dual}} \oplus P_i \quad (5)$$

where D_i is the dialog history up to the i th turn. $\text{PREFIX}_{\text{dual}}$ is the dual-prompt prefix. It indicates the task involves generating either state values or slot types. $P_i = \{P_{i1}, P_{i2}, \dots, P_{in}\}$ indicates the selected n subprompts. These subprompts are concatenated with the dialog history and the dual-prompt prefix to form the prompt input for the T5 model. The equation of distance loss $\mathcal{L}_{\text{dis}}$ between the dialog vector x_i and each subprompt is shown below

$$\mathcal{L}_{\text{dis}} = \sum_{k_i \in K_i} \gamma(x_i, k_i) \quad (6)$$

where $\gamma(\cdot)$ calculates the cosine similarity distance between the dialog vector x_i and the key vector k_i . K_i represents the set of key vectors corresponding to the selected subprompts in the i th turn. By selecting the most helpful subprompt embeddings for the task, the key vectors are dynamically optimized.

To ensure the selection of the most task-relevant subprompts, the key vectors k_t are optimized through back-propagation. During training, these key vectors participate in the forward pass via cosine similarity calculations that directly influence the loss function. In the backward pass, gradients are propagated to the prompt pool, enabling end-to-end optimization of the key vectors. Notably, the BERT encoder is frozen during training, and only the key vectors in the prompt pool are updated. Moreover, similarity weights are dynamically modulated using a frequency penalty term to avoid over-selection of frequent prompts and to improve the diversity of prompt selection.

2) *RE Feedback*: Dual-prompts adaptive to the dialog context are obtained through DD construction. The dual-generation task loop is used to measure the difference between generated dual slots and the original slots. The semantic

distance between slots and values is reduced during training, enabling the generation of matching slots. The dual-generation task loop is shown in Fig. 5.

In Fig. 5, the slot prompts guide the T5 model to generate state values corresponding to the inner loop's primal task. The value prompts further guide the model in generating dual slots by utilizing the generated values corresponding to the inner loop's dual task, forming a dual-task loop. Meanwhile, state value and slot generation are considered independent tasks. The equation of the state value generation task loss $\mathcal{L}_v$ is shown below

$$\mathcal{L}_v = - \sum \log P(v | D_i, M_i(s)) \quad (7)$$

where $P(v | D_i, M_i(s))$ is the probability prediction of the output real state value. D_i denotes the dialog history up to the i th turn. $M_i(s)$ represents the prompt sequence for the given slot s , v denotes the ground-truth state value.

Similarly, the equation of the loss $\mathcal{L}_s$ for the slot generation task is shown below

$$\mathcal{L}_s = - \sum \log P(s | D_i, M_i(v)) \quad (8)$$

where $P(s | D_i, M_i(v))$ is the probability prediction of the output real slot. $M_i(v)$ represents the prompt sequence for the given state value v , and s is the true slot label.

During the dual-generation task loop training, the RE is minimized to strengthen the semantic mapping relationship between values and slots. To obtain the RE E , we compute the cross-entropy between the predicted dual slot tokens and the original slot tokens. The equation of E is shown below

$$E = \frac{1}{m} \sum_{j=1}^m \text{CE}(S_j, \hat{S}_j) \quad (9)$$

where $\text{CE}(\cdot)$ denotes the cross-entropy loss. S_j represents the j th generated slot of individual task, and $\hat{S}_j$ is the predicted slot generated from the dual path. m is the total number of slots considered.

In summary, by combining the loss functions of the two independent tasks and the RE, the equation of the overall loss $\mathcal{L}_{\text{total}}$ is shown below

$$\mathcal{L}_{\text{total}} = \lambda_0 \mathcal{L}_s + \lambda_1 \mathcal{L}_v + \lambda_2 E + \lambda_3 \mathcal{L}_{\text{dis}} \quad (10)$$

where $\mathcal{L}_v$ and $\mathcal{L}_s$ represent the individual task loss functions for state value generation and slot generation. $\lambda_0, \lambda_1, \lambda_2$, and λ_3 are the weight parameters for each loss term used to balance the focus on different tasks during training. During the closed-loop training of the dual generation task, the RE serves as

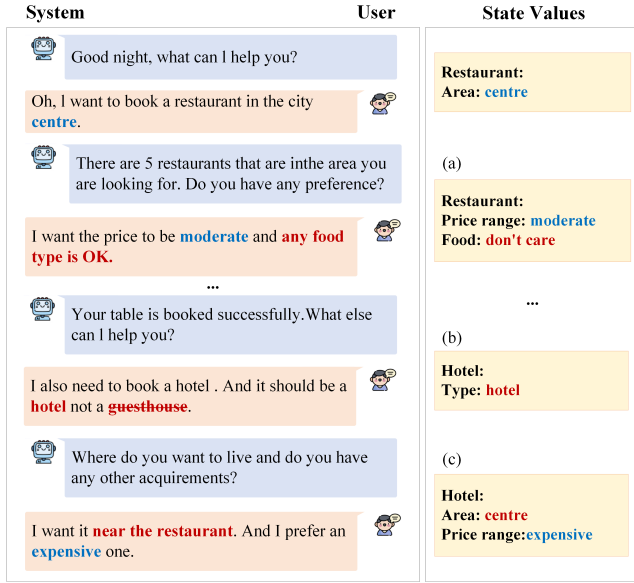


Fig. 6. Example of belief state generation.

a feedback signal to constrain the T5 model. It dynamically adjusts the slot type generation and dual-prompt construction processes.

Based on the duality of the tasks, the RE feedback method designs a dual-task closed loop for state value generation and slot generation. During the training process, the distance between the dual slot and the actual slot is reduced. By enhancing the semantic understanding of slot and state value relationships, the few-shot DST task accuracy is improved.

C. Belief State Generating

After the dual-prompt learning fine-tuning, the DDP-DST model generates belief states. It mainly consists of two stages: state value generation and slot generation.

In the state value generation stage, the T5 model is trained by the DS method to generate state values based on the dialog context. In the slot generation stage, the T5 model is fine-tuned with dual-prompt learning. Combined with the state values generated in the previous stage and the dialog history, the T5 model generates the corresponding slot. The belief state updates when the current slot-value pair differs from the last round. A new belief state is added if the slot and value are not presented in the list. An example of belief state generation is shown in Fig. 6.

In Fig. 6, the DDP-DST model labels the irrelevant slots “Price range” and “Parking” as “don’t care,” ignores it in the belief state list. When multiple entities appear in the dialog context, the DDP-DST model distinguishes the state value “hotel,” representing the user’s intent and updating the state. Some state values, like “cambridge,” need to be inferred and generated by the DDP-DST model based on the context.

The DDP-DST model leverages the DS state value generation method to iteratively learn state value features from two perspectives. The semantic understanding ability is enhanced. Dual-task prompts are generated through the dual-prompt learning fine-tuning method to guide the model in completing

the closed-loop training process. By obtaining matched slots, the belief states are updated.

In summary, the training algorithm steps for the DDP-DST model are described as follows.

Algorithm 1 DDP-DST Model Training

Input: Datasets D , Learning rate Lr , Number of selected sub-prompts n , Batch-size Bs , Self-training Iterations It , Number of training rounds Ep , The initial DDP-DST model parameters (θ)

Output: The DDP-DST model ($\hat{\theta}$)

```

1   $T_{\text{few}} \leftarrow \text{Preprocess}(D)$ 
2  Compute the augmented dataset  $T_{\text{EDA}}$  and  $T_{\text{Crop}} \leftarrow T_{\text{few}}$ 
3   $\theta_{\text{EC-DualT5}} \leftarrow \arg \min_{\theta} \mathcal{L}_{\text{Self}}(T_{\text{EDA}}, T_{\text{Crop}})$ 
4  for  $i$  in range  $(1, It)$  do
5     $\hat{Y}_{\text{EDA}}, \hat{Y}_{\text{Crop}} \leftarrow \text{T5-large}(T_{\text{few}}, \theta_{\text{EC-DualT5}})$ 
6     $\hat{Y}_{\text{fused}} \leftarrow \text{Fuse}(\hat{Y}_{\text{eda}}, \hat{Y}_{\text{crop}})$ 
7    Build pseudo label set  $T_{\text{pseudo}} \leftarrow \hat{Y}_{\text{fused}}$ 
8    for  $e$  in range  $(1, Ep)$  do
9       $\text{loss}_T \leftarrow \text{Self-training Loss}(T_{\text{pseudo}})$ 
10      $\tilde{\theta} \leftarrow \theta - \eta \cdot \nabla \text{loss}(\theta)$ 
11      $\theta_{\text{EC-DualT5}} \leftarrow \alpha \cdot \tilde{\theta} + (1 - \alpha) \cdot \theta_{\text{EC-DualT5}}$ 
12      $e \leftarrow e + 1$ 
13   end for
14    $i \leftarrow i + 1$ 
15 end for
16 Initial the prompts and keys of prompt pool  $P, K$ 
17 for  $e$  in range  $(1, Ep)$  do
18    $x \leftarrow \text{Compute the context vector}(T_{\text{few}})$ 
19   Select  $n$  keys and sub-prompts  $K_n, P_n \leftarrow x, P, K$ 
20   Generate primal task prompt  $M_v$  and dual task prompt  $M_s \leftarrow P_n$ 
21    $\hat{V} \leftarrow \text{T5-large}(M_v, T_{\text{few}})$ 
22    $S \leftarrow \text{T5-large}(M_s, T_{\text{few}})$ 
23   Generate the dual slot  $\hat{S} \leftarrow \text{T5-large}(M_s, \hat{V})$ 
24    $\text{loss}_d \leftarrow \text{Sub-prompt Distance Loss}(x, K_n)$ 
25    $\text{loss}_{\text{total}} \leftarrow \text{Reconstruction Loss}(\hat{S}, S) +$ 
26      $\text{Primal-task Loss}(\hat{V}, M_v) + \text{Dual-task Loss}(S, M_s)$ 
27    $\hat{\theta} \leftarrow \tilde{\theta} - \eta \cdot \nabla \text{loss}(\tilde{\theta})$ 
28    $e \leftarrow e + 1$ 
29 end for
30 return The DDP-DST Model ( $\hat{\theta}$ )
```

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Dataset

The effectiveness of the DDP-DST model is evaluated on the MultiWOZ 2.1 dataset [6]. MultiWOZ 2.1 is a multidomain, multiturn ToD dataset containing 10 438 dialogs across seven domains. We use the official data splitting of the MultiWOZ 2.1 dataset. Following existing work [10], we keep five domains (Attraction, Hotel, Restaurant, Taxi, Train) because the other two domains only appear in the training set.

To demonstrate the few-shot performance of our model, we randomly sample 1%, 5%, 10%, and 25% of the labeled training data from the MultiWOZ 2.1 dataset. The model is trained sequentially under each labeled data ratio, and evaluated on the full validation and test sets.

TABLE I
EXPERIMENTAL HYPERPARAMETER SETTINGS

Num	Hyper Param	Value	Notes
1	Lr	5e-5	Learning Rate
2	Bs	2	Batch Size
3	Epoch	15	Training Epochs
4	Seed	42	Random Seed
5	Top-n	3	Number of selected sub-prompts
6	It	2	Self-training Iterations

B. Implementation Details

All experiments are conducted on a single NVIDIA A100 GPU with 40 GB of VRAM. The DDP-DST model is implemented using Python 3.8.10, PyTorch 1.10.0, and CUDA 11.3. T5-large serves as the backbone model, which contains approximately 770 million parameters. For optimization, we employ the AdamW optimizer with a learning rate of lr, combined with a linear learning rate scheduler with 1000 warmup steps. A dropout rate of 0.1 is applied to mitigate overfitting. Gradient clipping with a maximum norm of 1.0 is applied to enhance training stability.

To ensure reproducibility, we fix the random seed to 42 and report the average performance over three independent runs with different seeds. The detailed hyperparameter configurations are summarized in Table I.

C. Evaluation Metrics

Inspired by the work in [10], turn level accuracy (TLA) and joint goal accuracy (JGA) are used as the evaluation metrics for the DDP-DST model. TLA refers to the ratio of dialog turns where state values are correctly predicted. It is used to evaluate the accuracy of state value prediction. JGA refers to the ratio of dialog turns where both slot types and state values are correctly predicted. It is used to measure the overall accuracy of belief state prediction.

The equation of TLA is shown below

$$TLA = \frac{N_{\text{value}}}{T} \quad (11)$$

where N_{value} represents the number of dialog turns that the state values are correctly predicted. T is the total number of turns.

The equation of JGA is shown below

$$JGA = \frac{\sum B_i}{T}, \quad B_i = \begin{cases} 1, & \hat{Y}_i = Y_i \\ 0, & \hat{Y}_i \neq Y_i \end{cases} \quad (12)$$

where B_i indicates whether all slot types and state values in turn i th are predicted correctly. Y_i denotes ground truth. $\hat{Y}_i$ is the set of predicted belief states.

D. Baselines

To evaluate the performance of the proposed DDP-DST model, we select a number of models to serve as baselines for the few-shot DST task. These models are briefly presented in the following according to the publication year. The baseline models for comparison include.

- 1) *TRADE* [29]: This model uses slot-specific decoders with copy mechanisms to generate slot values without relying on predefined ontologies.
- 2) *BERT* [30]: Built on a Transformer architecture, it treats DST as a multiclass classification task over a predefined ontology and is fine-tuned using limited MultiWOZ data.
- 3) *Self-Sup* [31]: It enforces latent consistency under input perturbations and introduces a behavior modeling task to reconstruct utterances from predicted states.
- 4) *TOD-BERT* [30]: By incorporating masked language modeling and response contrastive learning, TOD-BERT captures contextual semantics within dialogs.
- 5) *SGPDST* [32]: SGPDST uses domain and slot information from task-dependent patterns as prompts. Combined with slot descriptions derived from dialog contexts and database documents, it guides the T5 model in tracking dialog states.
- 6) *SF-DST* [33]: It leverages self-feeding belief state inputs to enhance multiturn contextual understanding. A binary classification task is employed to determine whether a slot is mentioned in the dialog.
- 7) *DS2* [4]: DS2 transforms DST into a dialog summarization task. By training a template-based text-to-text language model, synthesized dialog summaries are used to track the dialog states.
- 8) *IC-DST* [17]: IC-DST treats DST as a text-to-SQL problem, representing the dialog state as an SQL query. Additionally, a novel retrieval method is introduced to select relevant context examples.
- 9) *SM2* [19]: Meta in-context learning is employed to train the model, compressing long dialog histories and introducing a novel loss function to optimize the exemplary retrieval model.
- 10) *SynthDST* [35]: SynthDST leverages large language models (LLMs) to create coherent dialogs with DST annotations using only the dialog schema and hand-crafted templates. It enables retrieval-based few-shot prompting without requiring human-annotated data.
- 11) *CombiSearch* [36]: It combines diverse example pool construction with combinatorial reranking, scoring examples by their joint contribution to overall DST accuracy for optimized in-context selection.
- 12) *SVAG* [10]: It uses a self-training method to improve the accuracy of state value generation and employs inverse prompt learning to generate matching slots.
- 13) *EDZ-DA* [34]: It introduces an easy-to-difficult zero-shot data augmentation strategy based on LLMs. The method iteratively plans, generates, and complicates synthetic dialogs to improve low-resource DST performance.
- 14) *FaS-DST* [23]: It decomposes DST into three subtasks: generating candidate summaries, extracting dialog states, and selecting the most accurate summary. A contrastive learning-based scorer is designed to enhance the faithfulness and factuality of selected summaries.

Note that the performance results of all baselines are available in existing literature ([29]; [30]; [31]; [32]; [33]; [4]; [17]; [19]; [35]; [23]; [10]; [34]; [36]), so we directly

TABLE II
PERFORMANCE COMPARISON OF BENCHMARK MODELS

Models	Parameter Size	External Data	MultiWOZ 2.1 JGA(%)			
			1%	5%	10%	25%
TRADE [29]	<1B	No	10.4	27.7	32.6	38.5
BERT [30]		No	6.4	19.6	32.9	40.8
Self-Sup [31]		No	19.5	30.6	34.5	40.2
TOD-BERT [30]		ToD Data	9.9	28.6	39.5	44.3
SGPDST [32]		No	32.1	43.1	46.9	-
SF-DST [33]		No	28.4	39.4	44.6	-
DS2 [4]		Summary Data	33.8	44.2	45.4	-
SVAG [10]		No	31.9	45.1	47.6	50.0
EDZ-DA[34]		LLM-generated Data	37.3	45.0	-	-
FaS-DST [23]		Summary Data	36.3	46.4	48.4	-
DDP-DST (Our Method)		No	34.6	47.1	48.9	51.9
IC-DST GPT-Neo 2.7B [17]	<10B	No	16.7	26.9	31.7	-
IC-DST CodeGen 2.7B [17]		No	20.7	29.6	33.8	-
SM2-3B [19]		Labeled ToD Data	38.1	39.9	39.8	-
SM2-11B [19]	>10B	Labeled ToD Data	38.4	44.6	46.0	-
IC-DST Codex-davinci 175B [17]		No	43.1	47.1	48.7	-
SynthDST [35]		LLM-generated Data	45.8	44.9	45.4	-
CombiSearch [36]		No	45.8	47.7	-	-

Bold indicates the model with the best results in the dataset.

reuse the reported results from their articles for a fair comparison.

E. Main Results

As shown in Table II, the DDP-DST model achieves competitive performance in the JGA metric on the MultiWOZ 2.1 dataset without incorporating any additional datasets. Compared to the state-of-the-art (SOTA) model FaS-DST with fewer than 1 B parameters, DDP-DST achieves performance gains of 0.7% and 0.5% under the 5% and 10% few-shot settings, respectively. Moreover, compared with the best-performing model SVAG in the 25% few-shot setting, DDP-DST achieves a further 1.9% improvement in JGA.

Among models with fewer than 10 B parameters, including IC-DST GPT-Neo 2.7 B, IC-DST CodeGen 2.7 B, and SM2-3B, DDP-DST yields average improvements of 9.4%, 15.0%, and 13.8% under the 1%, 5%, and 10% few-shot settings, respectively. Although CombiSearch [36] achieves competitive results in the 1% and 5% settings, its backbone model, GPT-3.5-Turbo, contains approximately 175 B parameters, making it substantially larger in scale than DDP-DST. Furthermore, the original article reports a 50.0% JGA when trained on the full MultiWOZ 2.1 dataset, which remains lower than DDP-DST's 51.9% JGA obtained using only 25% of the data.

The DDP-DST model demonstrates a significant advantage in JGA, highlighting its superiority for few-shot DST. Specifically, DDP-DST constructs a DS framework that enables the model to learn both key semantic information and diversified syntactic structures, thereby enhancing its semantic reasoning capability. In addition, the dual-prompt fine-tuning strategy effectively reduces the semantic distance between slots and state values, strengthens slot-value alignment, and improves adaptability to diverse dialog scenarios. Consequently, DDP-DST can effectively learn dialog representations from limited samples, leading to consistently higher accuracy.

TABLE III
PERFORMANCE UNDER DIFFERENT PSEUDOLABEL CONFIDENCE THRESHOLDS (%)

P_r	1%		5%		10%		25%	
	TLA	JGA	TLA	JGA	TLA	JGA	TLA	JGA
0.90	69.31	34.06	76.59	45.43	78.34	48.36	79.81	50.31
0.91	69.63	34.20	77.03	45.78	78.54	48.43	80.54	51.52
0.92	69.50	33.90	76.74	45.36	78.21	48.09	80.21	50.87
0.93	69.73	34.32	77.09	46.21	78.59	48.71	80.87	51.86
0.94	69.79	34.54	77.18	46.42	78.63	48.63	81.01	52.10
0.95	69.58	33.64	76.94	46.43	78.57	48.49	81.04	52.12
0.96	69.72	34.23	76.63	45.98	78.23	48.33	80.89	51.88
0.97	69.86	34.40	77.16	46.71	78.97	48.78	81.02	51.93
0.98	70.18	34.63	77.73	47.28	78.89	48.92	80.55	51.74
0.99	70.14	34.09	77.19	46.62	78.42	48.47	80.92	51.89

Bold indicates the model with the best results in the dataset.

F. Analysis of Pseudolabel Confidence Threshold

To empirically determine a suitable pseudolabel confidence threshold, we conduct a sensitivity analysis by varying the threshold P_r from 0.90 to 0.99 with an increment of 0.01. This threshold governs the minimum confidence required for a pseudolabel to be retained during training. Table III presents the performance results (TLA and JGA metrics) under different few-shot settings.

As shown in Table III, when the threshold is set extremely high (e.g., 0.99), the JGA scores drop consistently across all settings due to overly conservative filtering, which limits the number of retained pseudolabels and reduces training diversity. On the other hand, when the threshold is too low (e.g., 0.90–0.92), although more pseudolabels are included, the presence of less reliable labels likely introduces noise, also leading to suboptimal results.

Notably, a threshold of 0.98 consistently yields the best or near-best performance across most scenarios. Based on this empirical observation, we adopt 0.98 as the fixed pseudolabel confidence threshold in our method.

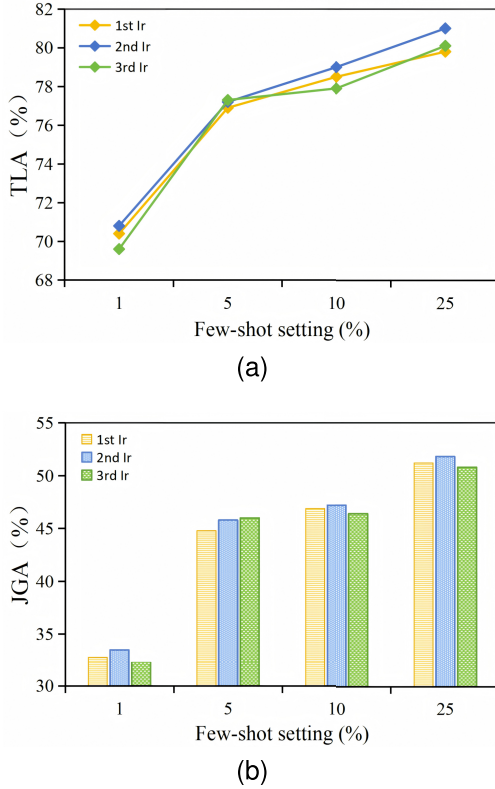


Fig. 7. Influence of self-training iterations on performance. (a) TLA across iterations. (b) JGA across iterations.

G. Analysis of DS Iterations

In the DS state value generation method, the number of self-training iterations has a significant impact on task accuracy. To investigate the optimal number of self-training cycles, we conducted iterative training for three iterations under the 1%, 5%, 10%, and 25% few-shot settings, respectively. The performance of the DDP-DST model on the MultiWOZ 2.1 dataset is illustrated in Fig. 7.

In Fig. 7, the DDP-DST model performs best when the self-training iterations are set to 2. When the self-training iterations are set to 1, the student model fails to fully learn the dialog knowledge from the unlabeled samples. Potential dialog feature information is likely overlooked. When the self-training iterations are set to 3, more pseudolabel noise is introduced. The training of the student model can be negatively affected, leading to lower state value generation performance. It indicates that setting the self-training iterations to 2 strikes a balance between learning dialog knowledge and introducing pseudolabel noise. It helps improve the quality of state value generation.

H. Analysis of Dual-Teacher Enhancement Strategy

To validate the effectiveness of the dual-teacher enhancement strategy, we employ T5, Crop-T5, EDA-T5, and EC-DualT5 as teacher models. Self-training is performed for two iterations under the 1%, 5%, 10%, and 25% few-shot settings. The performance of the DDP-DST model on the MultiWOZ 2.1 dataset is illustrated in Fig. 8.

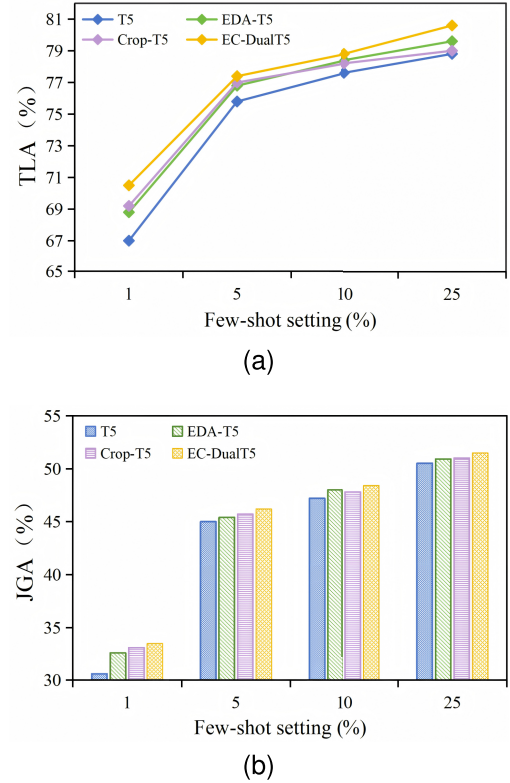


Fig. 8. Comparison of teacher models on performance. (a) TLA across different teacher models. (b) JGA across different teacher models.

In Fig. 8, under the same iterations, the dual-teacher model EC-DualT5 yields the best self-training performance. When using a single enhanced teacher for self-training, the StudentT5 learns pseudolabel knowledge from a single enhanced perspective. This results in higher dialog state generation accuracy compared to the original teacher model T5. When the EC-DualT5 model is used for self-training, the EDA-T5 and Crop-T5 teachers alternately activate to generate pseudolabels. After filtering, the StudentT5 learns dialog features with enhanced perspectives from both views. It effectively improves the state value generation quality of the DDP-DST model, achieving optimal performance in both TLA and JGA metrics.

I. Standard Deviation

To evaluate the stability of the DDP-DST model, we conducted experiments on the MultiWOZ 2.1 dataset using three random seeds. The experimental results in Table IV reflect the JGA and TLA scores of the models throughout the DS process.

As shown in Table IV, the StudentT5 consistently achieves the best performance across all few-shot settings, particularly during the second iteration of training. This indicates that the student model effectively integrates and distills knowledge from both teachers, leading to improved accuracy and more stable performance.

The teacher models also benefit from the iterative feedback. When comparing Iteration 2 to Iteration 1, both Crop-T5 and EDA-T5 exhibit noticeable improvements in JGA and TLA scores, accompanied by reduced standard deviations. For

TABLE IV
EXPERIMENTAL RESULTS OF STANDARD DEVIATION
ON MULTIWOZ 2.1 DATASET

Iterations	Model	Metrics	1%	5%	10%	25%
Iteration 1	Crop-T5	JGA(%)	31.83±0.24	43.77±0.18	46.51±0.41	49.61±0.22
		TLA(%)	67.41±0.14	75.19±0.51	77.02±0.28	78.73±0.34
	EDA-T5	JGA(%)	31.58±0.11	43.95±0.65	46.64±0.27	49.70±0.23
		TLA(%)	67.76±0.52	75.17±0.42	77.09±0.06	78.68±0.26
	StudentT5	JGA(%)	33.28±0.09	44.97±0.34	48.36±0.19	51.41±0.12
		TLA(%)	69.71±0.06	76.70±0.28	78.62±0.09	79.76±0.20
Iteration 2	Crop-T5	JGA(%)	33.94±0.18	45.50±0.41	47.86±0.20	51.14±0.29
		TLA(%)	69.93±0.04	76.66±0.16	78.18±0.10	79.69±0.31
	EDA-T5	JGA(%)	33.90±0.28	45.60±0.49	48.12±0.09	51.02±0.16
		TLA(%)	69.88±0.24	76.63±0.78	78.35±0.11	79.85±0.33
	StudentT5	JGA(%)	34.61±0.19	47.07±0.23	48.90±0.17	51.85±0.12
		TLA(%)	70.33±0.18	77.70±0.38	79.11±0.17	80.74±0.20

Note: The number after the "±" sign denotes the standard deviation of the model performance.

TABLE V
PAIRED *t*-TEST RESULTS BETWEEN DDP-DST AND THE BASELINE
MODEL SVAG UNDER DIFFERENT FEW-SHOT SETTINGS

Setting	Mean Diff	Std Diff	n	t	p-value
1%	2.82	0.48	3	10.15	0.010
5%	2.85	0.94	3	5.25	0.034
10%	1.33	0.24	3	9.50	0.011
25%	2.06	0.31	3	11.67	0.007

example, Crop-T5 improves from 31.83% to 33.94% JGA under the 1% setting, while EDA-T5 reaches 51.02% JGA in the 25% setting, surpassing its earlier performance.

Overall, these results validate the effectiveness of the DS framework with EMA-guided feedback. The relatively low variance in StudentT5's performance across different few-shot settings and iterations further confirms its robustness in handling few-shot DST tasks.

J. Statistical Significance Test

To assess the statistical significance of performance differences between DDP-DST and baseline models, a two-tailed paired *t*-test was conducted under consistent random seeds. The formula for the *t*-test is as follows:

$$t = \frac{\bar{d}}{s_d / \sqrt{n}} \quad (13)$$

where $\bar{d}$ denotes the mean of the paired differences between DDP-DST and baseline results, s_d is the standard deviation, and n is the number of paired samples. The null hypothesis (H_0) assumes that there is no significant difference between DDP-DST and baseline performance. The alternative hypothesis (H_1) states that there is a statistically significant difference. If the two-tailed *p*-value is less than 0.05, the null hypothesis is rejected, indicating that the observed performance improvement of DDP-DST is statistically significant. The specific calculations are shown in Table V.

As shown in Table V, all the *p*-values across the four few-shot settings are less than the significance threshold of 0.05. This allows us to reject the null hypothesis and conclude that the performance improvements of the DDP-DST model over the baseline model SVAG are statistically significant.

TABLE VI
COMPARISON OF LOSS FUNCTIONS FOR RE (%)

Loss Function	TLA	JGA
BASE	90.17	74.89
Contrastive Loss	90.83	75.62
Cosine Embedding	91.44	77.90
L2 Loss	90.48	75.23
Cross Entropy	91.85	78.38

Bold indicates the model with the best results in the dataset.

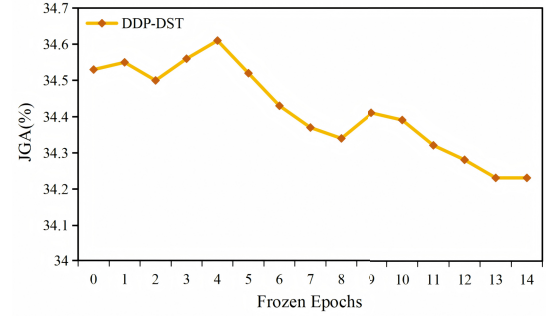


Fig. 9. JGA performance under different freezing epochs.

K. Analysis of Reconstruction Loss Definition

In order to identify more semantically aligned distance metrics for RE feedback, we conducted an ablation study comparing different loss functions. The test set performance of the DDP-DST model trained under the 1% setting on the MultiWOZ 2.1 dataset is presented in Table VI.

The results in Table VI demonstrate that the choice of reconstruction loss significantly affects model performance. These results are obtained by directly evaluating slot prediction accuracy based on the ground-truth slot values in the test set, allowing us to isolate the impact of the reconstruction loss function itself.

Among the evaluated options, Cross Entropy achieves the best overall performance, with 91.85% TLA and 78.38% JGA. This suggests that treating slot reconstruction as a token-level classification task offers more effective supervision, especially under low-resource conditions.

L. Analysis of Delayed Application of the Reconstruction Loss

To investigate whether deferring the application of the reconstruction loss can enhance training stability and convergence, we conduct an ablation study in which the reconstruction loss is frozen for the initial 0–14 epochs. During this period, only the main DST loss guided the model update. Fig. 9 presents the JGA achieved under each freeze configuration.

As shown Fig. 9, the results demonstrate that delaying the reconstruction loss has a noticeable impact on model performance. While the JGA starts at 34.53% with immediate reconstruction supervision, it gradually improves with longer freeze durations, peaking at 34.62% when the loss is introduced after 4 epochs. Beyond that point, the benefits plateau or slightly decline. This suggests that introducing the

TABLE VII
DUAL-TEACHER MODEL EC-DUALT5 TRAINING TIME PER
SELF-TRAINING ITERATION (MIN)

Iterations	Models	1%	5%	10%	25%
Iteration 1	Crop-T5	69	224	391	755
	EDA-T5	83	253	405	733
	StudentT5	311	303	310	334
Iteration 2	StudentT5	336	294	303	350

TABLE VIII
LATENCY AND GPU MEMORY USAGE COMPARISON BETWEEN BASE AND
DDP-DST MODELS IN FEW-SHOT SETTINGS

Metric	BASE	DDP-DST
Average inference time (ms)	39.84	40.19
Sub-prompt retrieval time (ms)	—	0.11
Training GPU memory usage (MB)	27,991	39,889

reconstruction signal too early may disrupt the learning of core task semantics, whereas a short warm-up period allows the model to build a stable representation before jointly optimizing auxiliary objectives.

M. Analysis of Training Efficiency

In this section, we further analyze the cost-effectiveness of the proposed DDP-DST models. Specifically, we measured the training time of both teacher models and the student model across different self-training iterations, as shown in Table VII.

Table VII presents the training time of the Dual-teacher model EC-DualT5 across different few-shot settings and self-training iterations. Each self-training iteration includes independent training of the two teacher models, followed by training the student model using pseudolabel supervision. The StudentT5 model, which incorporates pseudolabeled guidance from both teachers to learn a unified representation, demonstrates a moderate and stable training time, ranging from 294 to 350 min.

It is important to note that the two teacher models are not retrained in Iteration 2. Instead, they are updated through an EMA mechanism using the parameters of the StudentT5. This design ensures that the teacher models evolve with the student without introducing any additional training cost. Therefore, even in multi-round self-training, the overall training cost remains practical, and the proposed framework scales efficiently with minimal increase in resource consumption.

N. Analysis of Computational Efficiency

To assess the computational efficiency of the proposed DDP-DST model, we analyze both the inference latency and GPU memory usage. Table VIII presents the average inference time per sample, subprompt retrieval time, and GPU memory consumption during training.

As shown in Table VIII, DDP-DST achieves an average inference time of 40.19 ms per sample, comparable to the baseline. Since three subprompts are selected per input during decoding, the subprompt retrieval adds only 0.33 ms per sample, resulting in minimal latency overhead.

TABLE IX
CROSS-DOMAIN JGA IN 1% FEW-SHOT SETTING (%)

Models	Attraction	Hotel	Restaurant	Taxi	Train
BASE	59.09	46.12	60.47	74.19	73.23
BASE w/DS	60.64	47.72	61.67	74.78	74.29
DDP-DST	61.34	48.24	61.98	74.28	74.52

Bold indicates the model with the best results in the dataset.

Although DDP-DST is parameter-efficient, the prompt pool and retrieval mechanism slightly increase GPU memory usage and training time. However, as shown in Table VIII, the inference-time overhead is minimal. This negligible cost, combined with notable performance gains, makes the tradeoff worthwhile and supports the model's practicality for real-world deployment.

O. Few-Shot Cross-Domain Adaptability

To validate the performance of the DDP-DST model in few-shot cross-domain dialog scenarios, it is first trained on data from four source domains and then fine-tuned using 1% data from the target domain. The performance of the DDP-DST model on the MultiWOZ 2.1 dataset is shown in Table IX.

Table IX shows the JGA metric results of three models, BASE, BASE w/DS, and DDP-DST, across five dialog domain scenarios. The DDP-DST model achieves the best performance in the Attraction, Hotel, Restaurant, and Train domains, with an average improvement of 1.45% compared to the baseline model SVAG. When the DS module is added, the DDP-DST model learns the latent features of key information enhancement and syntactic structure enhancement in each domain. The JGA metric improves across all five dialog domains. After adding the dual-prompt learning fine-tuning module, the DDP-DST model adapts to different domain dialog scenarios. By matching similar subprompts from the prompt pool based on the dialog context, the DDP-DST model further improves the accuracy of the few-shot DST task.

P. Analysis of Additional Dataset

To further evaluate the effectiveness and generalizability of the proposed DDP-DST model, we conducted additional experiments on the MultiWOZ 2.0 dataset. While MultiWOZ 2.1 serves as our primary benchmark due to its corrected annotations, MultiWOZ 2.0 remains a widely used version in prior DST research. The performance of the proposed DDP-DST model on the MultiWOZ 2.0 and MultiWOZ 2.1 datasets under different few-shot settings are presented in Table X.

As shown in Table X, DDP-DST achieves comparable performance on MultiWOZ 2.0 and MultiWOZ 2.1 at each few-shot level, with slightly better results on MultiWOZ 2.1. This is likely due to the cleaner annotations in MultiWOZ 2.1, which reduce label noise during training.

Overall, these results demonstrate the robustness and transferability of the DDP-DST framework across different versions of the benchmark dataset.

TABLE X

PERFORMANCE OF DDP-DST ON MULTIWOZ 2.0 AND MULTIWOZ 2.1 UNDER DIFFERENT FEW-SHOT SETTINGS (%)

Setting	MultiWOZ 2.0		MultiWOZ 2.1	
	TLA	JGA	TLA	JGA
1%	69.03	33.01	70.33	34.61
5%	76.10	45.66	77.70	47.07
10%	77.89	47.02	79.11	48.90
25%	78.63	50.32	80.74	51.85

TABLE XI

ZERO-SHOT JGA PERFORMANCE COMPARISON BETWEEN DDP-DST AND BASE MODEL (%)

Models	Attraction	Hotel	Restaurant	Taxi	Train
BASE	38.21	29.28	29.26	68.22	41.22
DDP-DST	41.24	31.62	31.33	68.49	44.32

Bold indicates the model with the best results in the dataset.

Q. Analysis of Zero-Shot DST

To evaluate the zero-shot generalization capability of our proposed DDP-DST model, we conduct experiments on five unseen domains from the MultiWOZ 2.1 dataset without providing any labeled samples from these domains during training. The JGA performance of DDP-DST compared to the BASE model SVAG under zero-shot setting is shown in Table XI.

As shown in Table XI, our proposed DDP-DST model consistently outperforms the baseline (BASE) across all five domains under the zero-shot setting. Specifically, DDP-DST achieves notable gains in Attraction, Hotel and Restaurant, reflecting its improved ability to generalize without any in-domain labeled examples. In the Taxi domain, while the improvement is modest, it still demonstrates stability and effectiveness. The largest relative gain is observed in the Train domain. These results highlight the strong generalization capacity of DDP-DST in transferring knowledge from source domains to unseen target domains, making it well-suited for practical zero-shot DST scenarios.

R. Comparison With LLM-Based Models on Few-Shot DST

To evaluate the effectiveness of our proposed DDP-DST model under few-shot conditions, we compare it with several LLMs, including ChatGLM-4-32b [37] and LLaMA-4-maverick.¹ We formulate DST as a slot-value extraction task conditioned on the dialog history.

We construct few-shot demonstrations by selecting representative annotated dialogs from the training set. Each LLM is evaluated with 1-shot, 5-shot, 10-shot, and 25-shot examples prepended to the prompt. The test set remains fixed across all methods to ensure fair comparison. All LLMs are evaluated on the same test set using a unified system prompt to extract slot-value pairs in JSON format based on dialog history.

To further evaluate the effectiveness of DDP-DST, we additionally include a comparison with LDST [38], a

TABLE XII

JGA PERFORMANCE COMPARISON OF LLMs AND DDP-DST ON FEW-SHOT DST (%)

Model	1%	5%	10%	25%
ChatGLM-4-32B	31.98	47.83	46.34	45.53
LLaMA-4-maverick	43.43	56.07	56.10	51.78
LDST	44.12	49.87	51.43	52.48
DDP-DST	34.61	47.07	48.90	51.85

TABLE XIII

ABLATION STUDY RESULTS OF THE DDP-DST MODEL (%)

Models	1%		5%		10%		25%	
	TLA	JGA	TLA	JGA	TLA	JGA	TLA	JGA
DDP-DST	70.11	34.61	77.73	47.07	78.84	48.90	80.54	51.85
w/o DD	70.11	34.09	77.73	46.53	78.84	48.54	80.54	51.61
w/o RE	70.11	34.23	77.73	46.79	78.84	48.68	80.54	51.52
w/o DS	67.28	32.31	76.13	45.31	77.66	47.90	78.95	50.23

representative LLM-based DST framework built upon the open-source LLaMA-7B model without external data augmentation. We retrain LDST with the officially released code and hyper-parameters on MultiWOZ 2.1 to guarantee an apples-to-apples comparison. The JGA is used as the evaluation metric. The results are summarized in Table XII.

As shown in Table XII, DDP-DST achieves 48.90% and 51.85% JGA in the 10% and 25% settings, respectively, surpassing ChatGLM-4-32B and narrowing the gap with LLaMA-4-Maverick while requiring significantly fewer computational resources. Compared with LDST, which is built upon the LLaMA-7B backbone and adopts assembled domain-slot instruction tuning with parameter-efficient adaptation, DDP-DST yields slightly lower results, reasonable given LDST's larger backbone and instruction-tuned design. Nonetheless, DDP-DST demonstrates a more consistent upward trend as the number of samples increases. Despite its smaller backbone, DDP-DST achieves competitive and scalable performance across few-shot conditions.

While instruction-tuned LLMs such as LLaMA-4-Maverick and ChatGLM-4-32B exhibit strong few-shot capabilities, their performance improvements tend to plateau as the number of examples increases, owing to the frozen inference nature of LLMs. In contrast, DDP-DST benefits from gradient-based optimization, enabling it to continuously adapt as more training samples become available. As the proportion of labeled data increases from 1% to 25%, DDP-DST achieves steady gains and ultimately surpasses ChatGLM-4-32B in higher-shot settings, demonstrating superior flexibility and efficiency under limited supervision.

S. Ablation Experiments

The DDP-DST model is primarily divided into the DS module, the DD construction module, and the RE feedback module. To verify the effectiveness of individual modules in the DDP-DST model, we conducted controlled experiments by selectively removing key components. Ablation experiments are conducted to validate the effectiveness of each module. The results on the TLA and JGA metrics are shown in Table XIII.

¹<https://huggingface.co/meta-llama/Llama-4-Maverick-17B-128E>

From Table XIII, it can be observed that removing any module from the model leads to a decline in performance. Specifically, as shown in Table XIII, the absence of the DS module leads to an average decrease of 1.8% in TLA and 1.67% in JGA. The absence of the DD module causes an average reduction of 0.42% in JGA compared to the results when all modules are included. When the RE module is removed, the JGA drops by an average of 0.30%. These results demonstrate that all three modules introduced in this article contribute positively to the few-shot DST task.

To validate the effectiveness of the DS component, we remove the dual-teacher setup. Instead, slot values are generated through two iterations of self-training using an unenhanced T5 single-teacher model. Experimental results show significant performance degradation across key evaluation metrics in all few-shot settings. The dual-teacher model EC-DualT5 provides key information enhancement and syntactic structure enhancement from two perspectives, effectively learning latent dialog features. By improving the quality of state value generation, the accuracy of DST is enhanced.

By removing dynamically generated subprompts and using only the remaining static prompts, we obtain the ablation results for the removal of the DD module. Experimental results indicate that the DD construction method improves the ability to adapt to different scenarios. By matching the key vectors from the prompt pool with the dialog context, the model generates subprompts that are directionally similar in the high-dimensional space. It helps the model understand the current dialog scenario.

In the ablation experiment for the RE module, we remove the RE from the loss function of the slot generation task. Experimental results indicate that the RE feedback method aids the model in understanding slot semantics, as there is a complex dependency relationship between state values and slots. By using RE to reduce the semantic distance between slots and values, the model can better learn the semantic mapping within the belief state.

Meanwhile, Table XIII demonstrates that the RE and DD modules can jointly improve the quality of slot generation. DD helps the model adapt to few-shot dialog scenarios. The RE feedback method enhances the understanding of slot semantics, laying a solid foundation for few-shot DST. It is worth noting that the performance gains from the two modules are not linearly additive, which may be attributed to the increased complexity.

The overall ablation results demonstrate that the DS, DD, and RE modules complement each other, collectively enhancing the performance of DDP-DST in tracking belief states in few-shot dialog scenarios.

T. Case Study

In this section, we present several cases generated by the baseline model SVAG and the proposed DDP-DST model from the MultiWOZ 2.1 test set. These cases are used to intuitively demonstrate the performance of the two models in the few-shot DST task. The results are shown in Table XIV.

In the first case from Table XIV, the user requests a taxi from the Ashley Hotel at 10:45. While the SVAG model

TABLE XIV
COMPARISON OF THE CASES TO ILLUSTRATE THE EFFECTIVENESS OF DDP-DST IN 1% FEW-SHOT SETTING

Dialogue ID: SNG01332.json	Turn: 0
Dialogue:	
User: I need a taxi to pick me up at ashley hotel to leave after 10:45.	
Ground truth: (taxi-leaveat: 10:45), (taxi-departure: ashley hotel)	
BASE: (taxi-leaveat: 10:45), (taxi-destination: ashley hotel) ✕	
BASE w/DS: (taxi-leaveat: 10:45), (taxi-destination: ashley hotel) ✕	
DDP-DST: (taxi-leaveat: 10:45), (taxi-departure: ashley hotel) ✓	
Dialogue ID: PMUL1002.json	Turn: 2
Dialogue:	
User: Hi, i would like some information on nightclubs located in the west please.	
System: My system is not pulling up any nightclub attractions in the west. Is there another attraction type or area you would be interested in searching?	
User: How about a college to visit in the same area?	
System: Magdalene college is located in the west and is free to visit. Would you like more information on it?	
User: What is the postcode there?	
Ground truth: (attraction-type: college), (attraction-name: magdalene college), (attraction-area: west)	
BASE: (attraction-type: college), (attraction-area: west) ✕	
BASE w/DS: (attraction-type: college), (attraction-name: magdalene college), (attraction-area: west) ✓	
DDP-DST: (attraction-type: college), (attraction-name: magdalene college), (attraction-area: west) ✓	

correctly identifies the state values, it mistakenly labels the Ashley Hotel as the destination. Although the BASE w/DS model added a DS module on top of SVAG, its improvement in slot generation is limited. The DDP-DST model introduces a DD construction strategy. This enables it to dynamically adapt to the context of dialog when generating slots. It also utilizes a RE signal feedback method. This method enhances the model's understanding of the relationship between slots and their corresponding values. For slots that are easily confused, such as "taxi-departure" and "taxi-destination," the DDP-DST model can correctly interpret the semantics of state value "ashley hotel," thereby generating more accurate slots.

In the second case, after no nightclubs are found in the west, the user switches to an attraction—Magdalene College—and enquires about its postcode. Both the SVAG model and the DDP-DST model correctly infer that the location of the attraction is still the west based on "in the same area." However, the SVAG model fails to utilize the context to deduce the new attraction name. After incorporating the proposed DS module in this study, the DS method is used to generate slot values. This method helps the model learn state value features from different perspectives, and enables it to understand that the attraction name inquired about the postcode is magdalene college. As a result, both the BASE w/DS and DDP-DST models can generate the state value "magdalene college." This validates the effectiveness of the DS module in improving the accuracy of slot value generation.

By comparing these cases, we observe that the DDP-DST model shows improved performance in few-shot DST and contextual understanding, offering a promising direction for further research and refinement.

V. CONCLUSION

Tracking the dialog state with limited data has become a key area of ToD research. This article proposes the dual-teacher and DDP-DST. By enhancing key dialog information and extending contextual syntactic diversity, the dual-teacher model EC-DualT5 is trained to generate pseudolabels from different perspectives. The student T5 learns the fused knowledge and updates the dual teachers' parameters through the EMA mechanism. State values are iteratively generated through self-training, improving the model's semantic comprehension ability. Then, the DDP-DST model perceives the dialog context and matches candidate subprompts from the prompt pool. State-related key vectors are selected to generate dual-task prompts. Feedback signals are established to complete the closed-loop training process. It allows the DDP-DST model to adapt to slot structures and semantic expressions dynamically, improving the accuracy of few-shot DST. Experimental results show that the DDP-DST model outperforms baseline models SGPDST, SF-DST, DS2 and SVAG in the JGA metric, with average improvements of 2.8%, 6.1%, 2.4%, and 2.0%, demonstrating its superior capability.

VI. LIMITATIONS

While DDP-DST demonstrates promising performance, certain aspects merit further exploration. The dual-teacher and dual-prompt designs enhance semantic representation but introduce additional complexity, which may impact deployment in resource-constrained settings. Moreover, reliance on the T5-large backbone may limit architectural flexibility. Future work will focus on more lightweight strategies and broader model integration to improve efficiency and generalization.

REFERENCES

- [1] Y. Wang, T. He, J. Mei, R. Fan, and X. Tu, "A stack-propagation framework with slot filling for multi-domain dialogue state tracking," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 1, pp. 1240–1254, Jan. 2022.
- [2] A. Kumar, P. Ku, A. Goyal, A. Metallinou, and D. Hakkani-Tur, "MA-DST: Multi-attention-based scalable dialog state tracking," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 5, pp. 8107–8114.
- [3] Y. Feng, Y. Wang, and H. Li, "A sequence-to-sequence approach to dialogue state tracking," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process.*, 2021, pp. 1714–1725.
- [4] J. Shin, H. Yu, H. Moon, A. Madotto, and J. Park, "Dialogue summaries as dialogue states (DS2), template-guided summarization for few-shot dialogue state tracking," in *Proc. Findings Assoc. Comput. Linguistics: ACL*, 2022, pp. 3824–3846.
- [5] Y. Yang, W. Lei, P. Huang, J. Cao, J. Li, and T. S. Chua, "A dual prompt learning framework for few-shot dialogue state tracking," in *Proc. ACM Web Conf.*, 2023, pp. 1468–1477.
- [6] M. Eric et al., "MultiWOZ 2.1: A consolidated multi-domain dialogue dataset with state corrections and state tracking baselines," in *Proc. 12th Int. Conf. Lang. Resour. Eval.*, 2019, pp. 422–428.
- [7] M.-R. Amini, V. Feofanov, L. Pauletto, L. Hadjadj, É. Devijver, and Y. Maximov, "Self-training: A survey," *Neurocomputing*, vol. 616, Feb. 2025, Art. no. 128904.
- [8] H. Zhang, J. Bao, H. Sun, H. Luo, W. Li, and S. Cui, "CSS: Combining self-training and self-supervised learning for few-shot dialogue state tracking," in *Proc. 2nd Conf. Asia-Pacific Chapter Assoc. Comput. Linguistics, 12th Int. Joint Conf. Natural Lang. Process.*, 2022, pp. 302–310.
- [9] J. Lee, C. Lee, Y. Kim, and G. G. Lee, "Tracking must go on: Dialogue state tracking with verified self-training," in *Proc. INTERSPEECH*, Aug. 2023, pp. 4678–4682.
- [10] M. Gu, Y. Yang, C. Chen, and Z. Yu, "State value generation with prompt learning and self-training for low-resource dialogue state tracking," in *Proc. Asian Conf. Mach. Learn.*, 2024, pp. 390–405.
- [11] X. Zheng, H. Cui, C. Xu, and X. Lu, "Dual teacher: A semisupervised cotraining framework for cross-domain ship detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5613312.
- [12] F. Huang, Z. Yao, and W. Zhou, "DTBS: Dual-teacher bi-directional self-training for domain adaptation in nighttime semantic segmentation," in *Proc. ECAI*, 2024, pp. 1084–1091.
- [13] H.-D. Xu, X.-L. Mao, P. Yang, F. Sun, and H. Huang, "Cross-domain coreference modeling in dialogue state tracking with prompt learning," *Knowledge-Based Syst.*, vol. 283, Jan. 2024, Art. no. 111189.
- [14] X. Dong, Y. Feng, Z. Lu, G. Shi, and X.-M. Wu, "Zero-shot cross-domain dialogue state tracking via context-aware auto-prompting and instruction-following contrastive decoding," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 8527–8540.
- [15] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, and S. Singh, "AutoPrompt: Eliciting knowledge from language models with automatically generated prompts," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 4222–4235.
- [16] X. Zhao, M. Yang, Q. Qu, and R. Xu, "Few-shot relation extraction with automatically generated prompts," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 3, pp. 4971–4983, Mar. 2025.
- [17] Y. Hu, C.-H. Lee, T. Xie, T. Yu, N. A. Smith, and M. Ostendorf, "In-context learning for few-shot dialogue state tracking," in *Proc. Findings Assoc. Comput. Linguistics*, 2022, pp. 2627–2643.
- [18] G. Qixiang et al., "Exploiting domain-slot related keywords description for few-shot cross-domain dialogue state tracking," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2022, pp. 2460–2465.
- [19] D. Chen, K. Qian, and Z. Yu, "Stabilized in-context learning with pre-trained language models for few shot dialogue state tracking," in *Proc. Findings Assoc. Comput. Linguistics*, 2023, pp. 1551–1564.
- [20] H. Liu, Y. Cai, Y. Zhou, Z. Ou, Y. Huang, and J. Feng, "Prompt pool based class-incremental continual learning for dialog state tracking," in *Proc. IEEE Autom. Speech Recognit. Understand. Workshop (ASRU)*, Dec. 2023, pp. 1–8.
- [21] M. M. Reimann, F. A. Kunneman, C. Oertel, and K. V. Hindriks, "A survey on dialogue management in human–robot interaction," *ACM Trans. Hum.-Robot Interact.*, vol. 13, no. 2, pp. 1–22, Jun. 2024.
- [22] C. Li, X. Liu, Y. Wang, D. Li, Y. Lan, and C. Shen, "Dialogue for prompting: A policy-gradient-based discrete prompt generation for few-shot learning," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 18481–18489.
- [23] S. Feng, H. Su, H. Xie, D. Wu, H. Huang, and W. Silamu, "Fact-aware summarization with contrastive learning for few-shot dialogue state tracking," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2024, pp. 12211–12215.
- [24] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2019.
- [25] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 6382–6388.
- [26] G. G. Sahin and M. Steedman, "Data augmentation via dependency tree morphing for low-resource languages," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 5004–5009.
- [27] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [28] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2019, pp. 4171–4186.
- [29] C.-S. Wu, A. Madotto, E. Hosseini-Asl, C. Xiong, R. Socher, and P. Fung, "Transferable multi-domain state generator for task-oriented dialogue systems," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 808–819.
- [30] C.-S. Wu, S. C. H. Hoi, R. Socher, and C. Xiong, "TOD-BERT: Pre-trained natural language understanding for task-oriented dialogue," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 917–929.
- [31] C.-S. Wu, S. C. H. Hoi, and C. Xiong, "Improving limited labeled dialogue state tracking with self-supervision," in *Proc. Findings Assoc. Comput. Linguistics*, 2020, pp. 4462–4472.

- [32] C.-H. Lee, H. Cheng, and M. Ostendorf, "Dialogue state tracking with a language model using schema-driven prompting," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 4937–4949.
- [33] J. Lee and G. G. Lee, "SF-DST: Few-shot self-feeding reading comprehension dialogue state tracking with auxiliary task," in *Proc. Interspeech*, Sep. 2022, pp. 1233–1237.
- [34] M. Gu and Y. Yang, "Plan, generate and complicate: Improving low-resource dialogue state tracking via easy-to-difficult zero-shot data augmentation," in *Proc. Findings Assoc. Comput. Linguistics ACL*, Aug. 2024, pp. 6988–7005.
- [35] A. Kulkarni, B.-H. Tseng, J. R. A. Moniz, D. Piraviperumal, H. Yu, and S. Bhargava, "SynthDST: Synthetic data is all you need for few-shot dialog state tracking," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics*, 2024, pp. 1988–2001.
- [36] H. Pyun, Y. Park, and Y. Jo, "Improving dialogue state tracking through combinatorial search for in-context examples," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics*, Jul. 2025, pp. 28694–28714.
- [37] A. Zeng et al., "ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools," 2024, *arXiv:2406.12793*.
- [38] Y. Feng, Z. Lu, B. Liu, L. Zhan, and X.-M. Wu, "Towards LLM-driven dialogue state tracking," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Dec. 2023, pp. 739–755.



Yuheng Li received the B.Eng. degree in information security from Jiangsu University, Zhenjiang, China, in 2021. She is currently pursuing the M.Sc. degree in computer science and technology with Hebei University of Engineering, Handan, China.

Her research interests include natural language processing, neural network systems, and dialog systems.



Di Wu received the M.Sc. degree in computer application technology from Hebei University of Engineering, Handan, China, in 2010, and the Ph.D. degree in computer application technology from Yanshan University, Qinhuangdao, China, in 2014.

From September 2021 to June 2022, she was a Visiting Scholar with Wuhan University, Wuhan, China. She is currently a Professor with the School of Information and Electrical Engineering, Hebei University of Engineering. Her research interests include data mining, natural language processing,

software security, and information retrieval.



Zhizhi Yu received the B.Eng. degree in new energy science and engineering from Zhejiang University of Water Resources and Electric Power, Hangzhou, China, in 2022. He is currently pursuing the M.Sc. degree in computer science and technology with Hebei University of Engineering, Handan, China.

His research interests include natural language processing, response generation, and dialog systems.